

Campus Ambassador CRM

Minimum viable product plan — scope, data model, build order, and open decisions.

The whole system in one paragraph

Ambassadors upload outreach sheets and log activities. A manager verifies that work. Verified work becomes points, points become two leaderboards — one for ambassadors, one for students — and every manual change to a score is recorded with a reason. Admins manage people and campuses; a super admin owns the scoring rules and the audit trail.

Scope: one app, four roles

The original brief asked for four separate panels. Four panels means four codebases to keep in sync, and roughly 80% of their screens are identical. Build one application with four permission levels instead — same routes, different visibility and different default filters. Super admin is an admin with the extra switches turned on.

Role	Sees	Can do
Ambassador	Only their own data	Upload sheets, log activities, view own rank and progress
Manager	Their assigned campuses	Verify uploads and activities, adjust scores, edit the student leaderboard
Admin	All campuses	Everything a manager can, plus create and assign ambassadors and managers
Super admin	Everything	Admin rights, plus delete records, edit scoring rules, export, and read the audit log

The decision that matters most

Scores are derived, never typed

Do not let anyone edit a score directly. Store every score as a derived total plus a list of adjustment events — who changed it, by how much, why, and when. The leaderboard is the sum of points from verified outreach plus the sum of adjustments. This costs nothing extra to build and saves the programme the first time an ambassador disputes their rank, which will happen in month one.

Data model

Seven tables carry the whole MVP.

Table	Key fields
users	id, name, email, role, campus_id, manager_id, status
campuses	id, name, city, manager_id
uploads	id, ca_id, file_url, row_count, status, reviewed_by, created_at
student_records	id, upload_id, ca_id, name, email, phone, year, course, interest, dedupe_hash
activities	id, ca_id, type, date, count, proof_url, status

Table	Key fields
score_adjustments	id, subject_type, subject_id, delta, reason, created_by, created_at
audit_log	actor_id, action, entity, before, after, timestamp

Both leaderboards fall out of this structure. The student board reads from student_records plus adjustments; the ambassador board reads from verified records and activities plus adjustments. Neither stores a score of its own.

The Excel pipeline

This is where ambassador CRMs usually break. Three steps, in this order:

- **Fixed template.** One downloadable .xlsx with locked headers: Name, Email, Phone, Year, Course, Interest. Reject anything that does not match rather than guessing at column mapping.
- **Preview before commit.** Parse in the browser and show a table with row-level errors — missing name, invalid email, duplicate. The ambassador fixes and re-uploads. Nothing reaches the database until they confirm.
- **Dedupe on commit.** Hash lowercased email plus phone. Duplicates within one ambassador merge silently; duplicates across two ambassadors are flagged for manager review — otherwise two people submit the same student list and the leaderboard becomes fiction.

Cap uploads at roughly 2,000 rows for v1.

Stack

Next.js with Supabase. That gives Postgres, authentication, file storage, and row-level security in one place, and row-level security is exactly the mechanism behind “an ambassador sees only their own rows.” Enforce permissions in the database, not in the interface — hiding a nav item is decoration, not security.

Concern	Choice
Framework and hosting	Next.js on Vercel
Database, auth, storage	Supabase (Postgres + row-level security)
Spreadsheet parsing	SheetJS, client-side
Tables and charts	TanStack Table, Recharts

Build order

Four weeks, sequenced so that the hardest thing to retrofit comes first.

	Focus	Deliverable
Week 1	Foundation	Authentication, roles, users and campuses tables, row-level security policies, empty dashboard shells per role. Get permissions right before anything else — retrofitting them later is painful.
Week 2	Excel pipeline	Downloadable template, upload to preview to commit flow, row-level validation, dedupe hashing, and the student records table with filters and search.

	Focus	Deliverable
Week 3	Scoring and leaderboards	Scoring engine driven by configurable weights, both leaderboards, the adjustment flow with mandatory reasons, and the ambassador progress page showing targets against actuals.
Week 4	Management surfaces	Manager verification queue, admin CRUD for people and campus assignment, super admin configuration page for weights and targets, audit log view, and data export.

Deliberately out of scope for v1

Email notifications, in-app messaging, a mobile app, bulk ambassador import, a custom report builder, badges and gamification beyond the leaderboard, and third-party integrations. Add them once real usage shows which parts of the product ambassadors actually open.

Interface direction

A light control-room aesthetic: white surfaces on a cool grey canvas, hairline rules instead of boxed cards, monospaced tabular numerals so points and deltas align column-wise, and colour reserved entirely for status. Neobrutalist emphasis — heavier blocks, one loud colour — is confined to the ambassador dashboard and the top of the leaderboard, where competitive energy helps. Everything a manager or admin touches stays quiet and dense.

Role	Hex	Used for
Canvas	#EEF1F5	Page background
Surface	#FFFFFF	Tables, cards, panels
Ink	#0F1720	Primary text and numerals
Accent	#2B5CFF	Selected nav, primary action
Status	#0E8F63 / #B26A00 / #C43B45	Verified, pending, rejected

Two decisions to make before development starts

1. Scoring weights

Write down now what a reached student is worth against an event, a workshop, a social post, and a referral. The entire product is a wrapper around this formula. If the weights are vague, the leaderboard feels arbitrary and ambassadors disengage within a cycle.

2. Student personal data

The system collects names, emails, and phone numbers of students who have not consented to sitting in a CRM. Decide who is allowed to export, add a consent line to the upload template, and set a retention period. Cheap to handle now, expensive to handle later.